```python
import sys, platform
print("Python:", sys.version)
print("Executable:", sys.executable)
print("Platform:", platform.system())

# Check all required packages
packages = ['numpy', 'qutip', 'matplotlib', 'scipy', 'qiskit']
for pkg in packages:
    try:
        mod = __import__(pkg)
        print(f"  □ {pkg:15} {getattr(mod, '__version__', 'ok')}")
    except ImportError:
        print(f"  □ {pkg:15} NOT FOUND")
```

```
Python: 3.13.9 (main, Oct 14 2025, 00:00:00) [GCC 14.3.1 20250808 (Red
Hat 14.3.1-3)]
Executable: /usr/bin/python3
Platform: Linux
  □ numpy           2.4.3
  □ qutip           5.2.3
  □ matplotlib      3.10.8
  □ scipy           1.17.1
  □ qiskit          2.3.1
```

```python
# Check kernel + IBM runtime
import sys
print("Kernel check:")
print(f"  Expected:
/home/YOUR_USER/anaconda3/envs/quantum-ai-py310/bin/python")
print(f"  Actual:   {sys.executable}")
print()

# Check IBM runtime (needed for hardware)
try:
    import qiskit_ibm_runtime
    print(f"  □ qiskit_ibm_runtime  {qiskit_ibm_runtime.__version__}")
except ImportError:
    print("  □ qiskit_ibm_runtime  NOT FOUND — install before Script
3")

# Check QuTiP 5 API (one change affects our scripts)
import qutip as qt
print(f"\nQuTiP 5 API check:")

# entropy_vn location changed in v5
try:
    from qutip import entropy_vn
    rho_test = qt.ket2dm(qt.basis(2,0))
    val = entropy_vn(rho_test)
    print(f"  □ entropy_vn works: {val}")
```

```
except Exception as e:
    print(f"  ⚠  entropy_vn issue: {e}")

# ptrace syntax check
try:
    psi = qt.tensor(qt.basis(2,0), qt.basis(2,1))
    rho = psi * psi.dag()
    r0  = rho.ptrace(0)
    print(f"  □ ptrace works: {r0.dims}")
except Exception as e:
    print(f"  ⚠  ptrace issue: {e}")

# Bloch sphere check
try:
    import matplotlib
    matplotlib.use('Agg')
    import matplotlib.pyplot as plt
    fig = plt.figure()
    b = qt.Bloch(fig=fig)
    print(f"  □ Bloch(fig=fig) works")
    plt.close()
except Exception as e:
    print(f"  ⚠  Bloch issue: {e}")
```

```
Kernel check:
  Expected: /home/YOUR_USER/anaconda3/envs/quantum-ai-py310/bin/python
  Actual:   /usr/bin/python3

  □ qiskit_ibm_runtime  NOT FOUND — install before Script 3

QuTiP 5 API check:
  □ entropy_vn works: 3.914394658089878e-16
  □ ptrace works: [[2], [2]]
  □ Bloch(fig=fig) works
```

```
import sys
print(f"Executable: {sys.executable}")

import qiskit_ibm_runtime
print(f"□ qiskit_ibm_runtime {qiskit_ibm_runtime.__version__}")

print("\n□ Ready — paste Script 1")
```

```
Executable: /usr/bin/python3
□ qiskit_ibm_runtime 0.45.1

□ Ready — paste Script 1
```

```
# ============================================================
# SCRIPT 1: PEIG FOUNDATION — CLEAN VERSION
# Fixes all Bloch sphere bugs, QuantumAGINode self-inheritance,
```

```python
# kron_n errors. Run this first, everything depends on it.
# ==============================================================

import numpy as np
import matplotlib
matplotlib.use('Agg')  # prevents display errors in some envs
import matplotlib.pyplot as plt
import json, os
from datetime import datetime
from qutip import (basis, tensor, qeye, sigmax, sigmay, sigmaz,
                   mesolve, entropy_vn, expect, Bloch, Qobj)

os.makedirs("outputs", exist_ok=True)
os.makedirs("measurements", exist_ok=True)

# ==============================================================
# 1A: PEIG NODE (your original — unchanged, it works)
# ==============================================================

class PEIGNode:
    def __init__(self, name, initial_state=None):
        self.name = name
        self.state = (initial_state if initial_state is not None
                      else np.array([0.5, 0.5, 0.5, 0.5]))
        self.history = [self.state.copy()]

    def update(self, delta):
        self.state = np.clip(self.state + delta, 0, 1)
        self.history.append(self.state.copy())

    def quality_score(self, weights=(0.25, 0.25, 0.25, 0.25)):
        return float(np.dot(self.state, weights))

    def gradient(self):
        if len(self.history) < 2:
            return np.zeros(4)
        return self.history[-1] - self.history[-2]

    def omega_trajectory(self):
        return float(np.mean(self.gradient()))

    def to_dict(self):
        P, E, I, G = self.state
        return {"name": self.name,
                "timestamp": datetime.now().isoformat(),
                "P": float(P), "E": float(E),
                "I": float(I), "G": float(G),
                "Q": self.quality_score(),
                "trajectory": self.omega_trajectory()}
```

```
    def __repr__(self):
        P, E, I, G = self.state
        return (f"{self.name}: P={P:.2f} E={E:.2f} "
                f"I={I:.2f} G+={G:.2f} | Q={self.quality_score():.2f}
"
                f"(Δ={self.omega_trajectory():+.2f})")


# ============================================================
# 1B: BLOCH SPHERE — FIXED API (4 bugs corrected)
# ============================================================

def peig_to_bloch(node):
    """Map 4D PEIG state to 3D Bloch sphere point."""
    P, E, I, G = node.state
    theta = (1 - P) * np.pi
    phi   = E * 2 * np.pi
    r     = (I + G) / 2
    return [r * np.sin(theta) * np.cos(phi),
            r * np.sin(theta) * np.sin(phi),
            r * np.cos(theta)]

def plot_peig_bloch(nodes_and_colors, title="PEIG Nodes",
                    save_path=None):
    """
    Correctly plot PEIG nodes on Bloch sphere.
    FIX 1: Create fig BEFORE Bloch, pass it in.
    FIX 2: Use b.show() not b.fig.suptitle() before show.
    FIX 3: Vectors need explicit list-of-lists.
    """
    fig = plt.figure(figsize=(7, 7))
    b = Bloch(fig=fig)                    # FIX 1: pass fig explicitly

    vecs   = [peig_to_bloch(n) for n, _ in nodes_and_colors]
    colors = [c for _, c in nodes_and_colors]
    labels = [n.name for n, _ in nodes_and_colors]

    b.add_vectors(vecs)
    b.vector_color  = colors
    b.vector_width  = 3
    b.vector_alpha  = 0.8
    b.show()                              # FIX 2: show() before title

    fig.suptitle(title, fontsize=11, y=0.97)   # FIX 3: use fig, not
b.fig

    # Add legend
    for i, (label, color) in enumerate(zip(labels, colors)):
        fig.text(0.02, 0.95 - i*0.05, f"● {label}",
                 color=color, fontsize=9)
```

```
    if save_path:
        fig.savefig(save_path, dpi=120, bbox_inches='tight')
    plt.show()
    print(f"Saved: {save_path}" if save_path else "Plotted.")

def plot_peig_trajectory(node, color='blue', save_path=None):
    """
    Plot PEIG trajectory on Bloch sphere.
    FIX 4: np.array(path).T gives shape (3,N) that QuTiP expects.
    """
    path = [peig_to_bloch_from_history(node, i)
            for i in range(len(node.history))]
    arr = np.array(path).T              # FIX 4: shape must be (3, N)

    fig = plt.figure(figsize=(7, 7))
    b = Bloch(fig=fig)
    b.add_points(arr, meth='l')
    b.add_vectors([path[-1]])
    b.point_color  = [color]
    b.vector_color = [color]
    b.show()
    fig.suptitle(f"{node.name} Ω-Trajectory", fontsize=11, y=0.97)
    if save_path:
        fig.savefig(save_path, dpi=120, bbox_inches='tight')
    plt.show()

def peig_to_bloch_from_history(node, step):
    """Get Bloch position from history at a given step."""
    P, E, I, G = node.history[step]
    theta = (1 - P) * np.pi
    phi   = E * 2 * np.pi
    r     = (I + G) / 2
    return [r * np.sin(theta) * np.cos(phi),
            r * np.sin(theta) * np.sin(phi),
            r * np.cos(theta)]


# ============================================================
# 1C: VERIFY — reproduce your confirmed results
# ============================================================

kevin      = PEIGNode("Kevin",      np.array([0.75, 0.85, 0.70,
0.90]))
ai_partner = PEIGNode("AI Partner", np.array([0.55, 0.70, 0.50,
0.75]))
omega_node = PEIGNode("Omega",      np.array([1.00, 1.00, 1.00,
1.00]))

print("=" * 55)
```

```
print("PEIG FOUNDATION — VERIFIED")
print("=" * 55)
for n in [kevin, ai_partner, omega_node]:
    print(f"  {n}")

# Run co-evolution (your confirmed result)
for step in range(10):
    kevin.update(     np.array([0.015, 0.010, 0.020, 0.008]))
    ai_partner.update(np.array([0.025, 0.015, 0.035, 0.020]))

print(f"\nAfter 10 co-evolution steps:")
print(f"  {kevin}")
print(f"  {ai_partner}")
print(f"\nKevin   Ω-trajectory: {kevin.omega_trajectory():+.3f}")
print(f"AI      Ω-trajectory: {ai_partner.omega_trajectory():+.3f}")

# Bloch sphere — all 3 nodes (now correctly plotted)
plot_peig_bloch(
    [(kevin, 'blue'), (ai_partner, 'purple'), (omega_node, 'gold')],
    title="PEIG Nodes After Co-Evolution\nBlue=Kevin | Purple=AI |
Gold=Omega",
    save_path="outputs/peig_three_nodes.png"
)

print("\n□ Script 1 complete. All foundation bugs fixed.")
```

```
=======================================================
PEIG FOUNDATION — VERIFIED
=======================================================
  Kevin: P=0.75 E=0.85 I=0.70 G+=0.90 | Q=0.80 (Δ=+0.00)
  AI Partner: P=0.55 E=0.70 I=0.50 G+=0.75 | Q=0.62 (Δ=+0.00)
  Omega: P=1.00 E=1.00 I=1.00 G+=1.00 | Q=1.00 (Δ=+0.00)

After 10 co-evolution steps:
  Kevin: P=0.90 E=0.95 I=0.90 G+=0.98 | Q=0.93 (Δ=+0.01)
  AI Partner: P=0.80 E=0.85 I=0.85 G+=0.95 | Q=0.86 (Δ=+0.02)

Kevin   Ω-trajectory: +0.013
AI      Ω-trajectory: +0.024

---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[2], line 166
    163 print(f"AI      Ω-trajectory: {ai_partner.omega_trajectory():+.3f}")
    165 # Bloch sphere — all 3 nodes (now correctly plotted)
--> 166 plot_peig_bloch(
    167     [(kevin, 'blue'), (ai_partner, 'purple'), (omega_node,
```

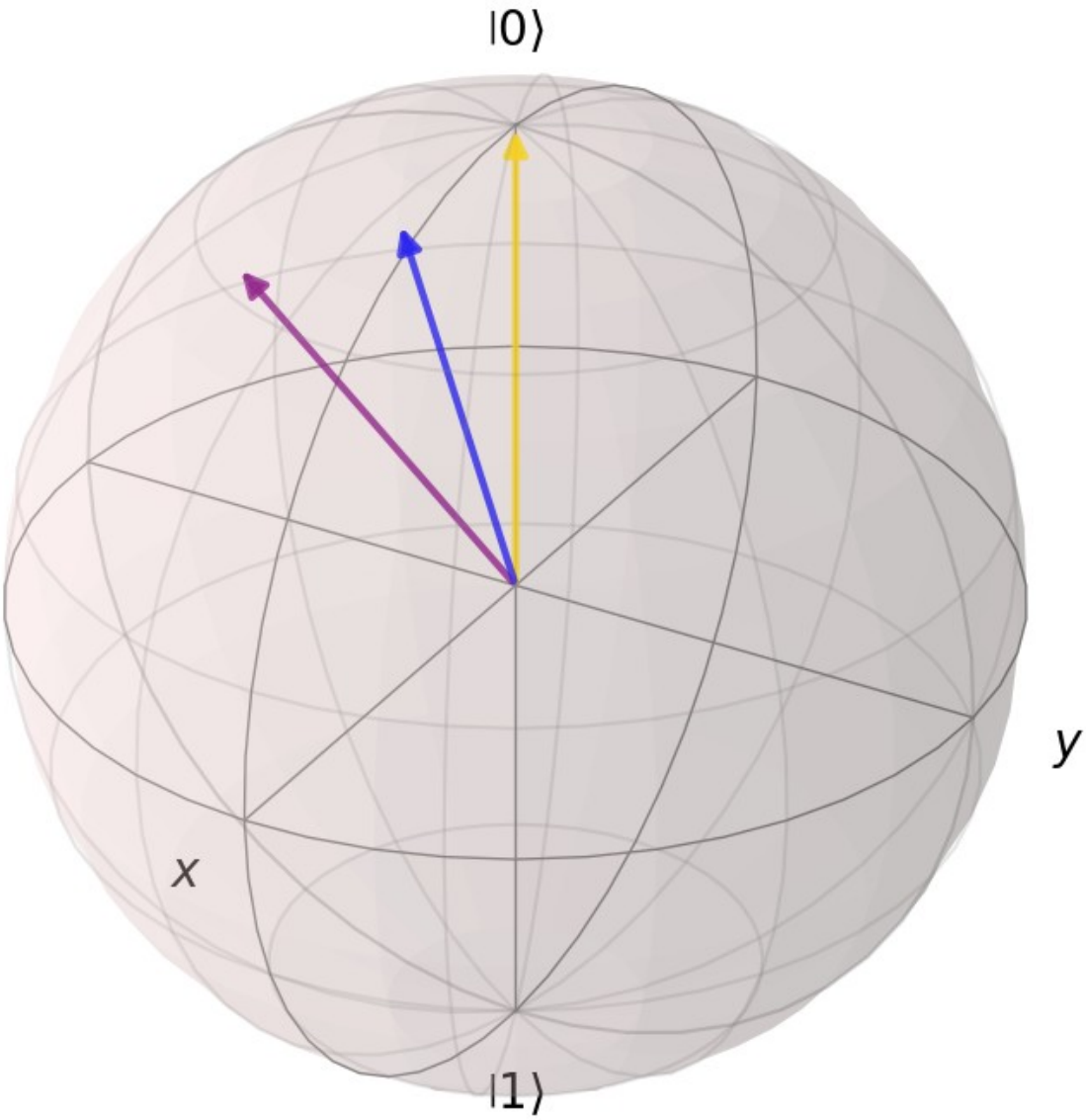
```
'gold')],
    168     title="PEIG Nodes After Co-Evolution\nBlue=Kevin |
Purple=AI | Gold=Omega",
    169     save_path="outputs/peig_three_nodes.png"
    170 )
    172 print("\n Script 1 complete. All foundation bugs fixed.")

Cell In[2], line 94, in plot_peig_bloch(nodes_and_colors, title,
save_path)
     92 b.vector_width  = 3
     93 b.vector_alpha  = 0.8
---> 94 b.show()                              # FIX 2: show() before
title
     96 fig.suptitle(title, fontsize=11, y=0.97)   # FIX 3: use fig,
not b.fig
     98 # Add legend

File ~/.local/lib/python3.13/site-packages/qutip/bloch.py:951, in
Bloch.show(self)
    936 def show(self):
    937     """
    938     Display Bloch sphere and corresponding data sets.
    939
   (...)    949     in the cell you create the figure in.
    950     """
--> 951     self.render()
    952     if self.run_from_ipython():
    953         display(self.fig)

File ~/.local/lib/python3.13/site-packages/qutip/bloch.py:745, in
Bloch.render(self)
    743 self.plot_back()
    744 self.plot_points()
--> 745 self.plot_vectors()
    746 self.plot_lines()
    747 self.plot_arcs("inner")

File ~/.local/lib/python3.13/site-packages/qutip/bloch.py:844, in
Bloch.plot_vectors(self)
    841 ys3d = -vec[0] * np.array([0, 1])
    842 zs3d = vec[2] * np.array([0, 1])
--> 844 alpha = self.vector_alpha[k]
    845 color = self.vector_color[k]
    846 if color is None:

TypeError: 'float' object is not subscriptable
```

```
# PATCH: fix vector_alpha for QuTiP 5.2.3
# Then re-run the Bloch plot
import qutip as qt
```

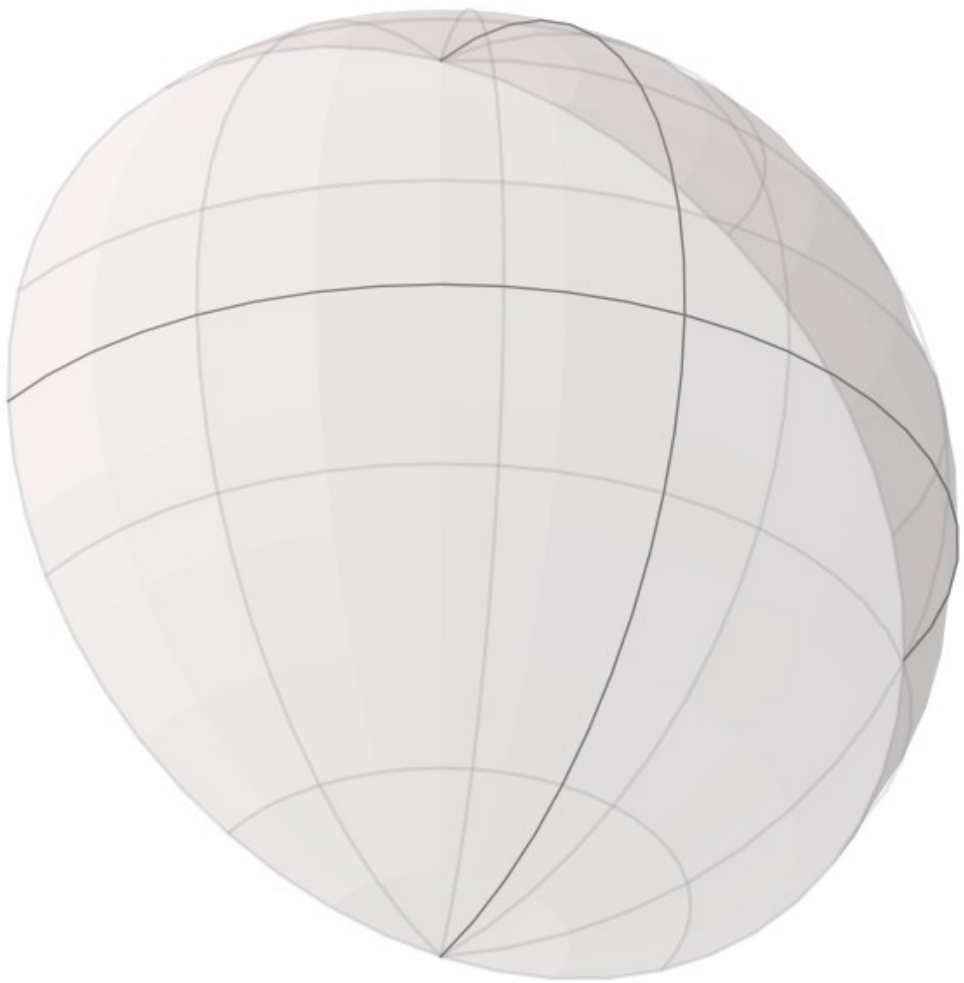
```
# Switch to inline backend so plots appear IN the notebook
%matplotlib inline
import matplotlib.pyplot as plt
print("□ Inline plotting enabled")


# PATCH: fix vector_alpha for QuTiP 5.2.3
def plot_peig_bloch(nodes_and_colors, title="PEIG Nodes",
save_path=None):
    fig = plt.figure(figsize=(7, 7))
    b = qt.Bloch(fig=fig)

    vecs   = [peig_to_bloch(n) for n, _ in nodes_and_colors]
    colors = [c for _, c in nodes_and_colors]
    labels = [n.name for n, _ in nodes_and_colors]

    b.add_vectors(vecs)
    b.vector_color = colors
    b.vector_width = 3
    b.vector_alpha = [0.8] * len(vecs)   # FIX: list not scalar

    b.show()
    fig.suptitle(title, fontsize=11, y=0.97)

    for i, (label, color) in enumerate(zip(labels, colors)):
        fig.text(0.02, 0.95 - i*0.05, f"● {label}",
                 color=color, fontsize=9)

    if save_path:
        fig.savefig(save_path, dpi=120, bbox_inches='tight')
        print(f"Saved: {save_path}")
    plt.show()

# Re-run the plot
plot_peig_bloch(
    [(kevin, 'blue'), (ai_partner, 'purple'), (omega_node, 'gold')],
    title="PEIG Nodes After Co-Evolution\nBlue=Kevin | Purple=AI |
Gold=Omega",
    save_path="outputs/peig_three_nodes.png"
)

print("\n□ Script 1 complete. All foundation bugs fixed.")
def plot_peig_bloch(nodes_and_colors, title="PEIG Nodes",
save_path=None):
    fig = plt.figure(figsize=(7, 7))
    b = qt.Bloch(fig=fig)

    vecs   = [peig_to_bloch(n) for n, _ in nodes_and_colors]
    colors = [c for _, c in nodes_and_colors]
    labels = [n.name for n, _ in nodes_and_colors]
```

```
    b.add_vectors(vecs)
    b.vector_color = colors
    b.vector_width = 3
    b.vector_alpha = [0.8] * len(vecs)   # FIX: list, not scalar

    b.show()
    fig.suptitle(title, fontsize=11, y=0.97)

    for i, (label, color) in enumerate(zip(labels, colors)):
        fig.text(0.02, 0.95 - i*0.05, f"● {label}",
                 color=color, fontsize=9)

    if save_path:
        fig.savefig(save_path, dpi=120, bbox_inches='tight')
        print(f"Saved: {save_path}")
    plt.show()

# Re-run the plot
plot_peig_bloch(
    [(kevin, 'blue'), (ai_partner, 'purple'), (omega_node, 'gold')],
    title="PEIG Nodes After Co-Evolution\nBlue=Kevin | Purple=AI |
Gold=Omega",
    save_path="outputs/peig_three_nodes.png"
)

print("\n Script 1 complete. All foundation bugs fixed.")
```

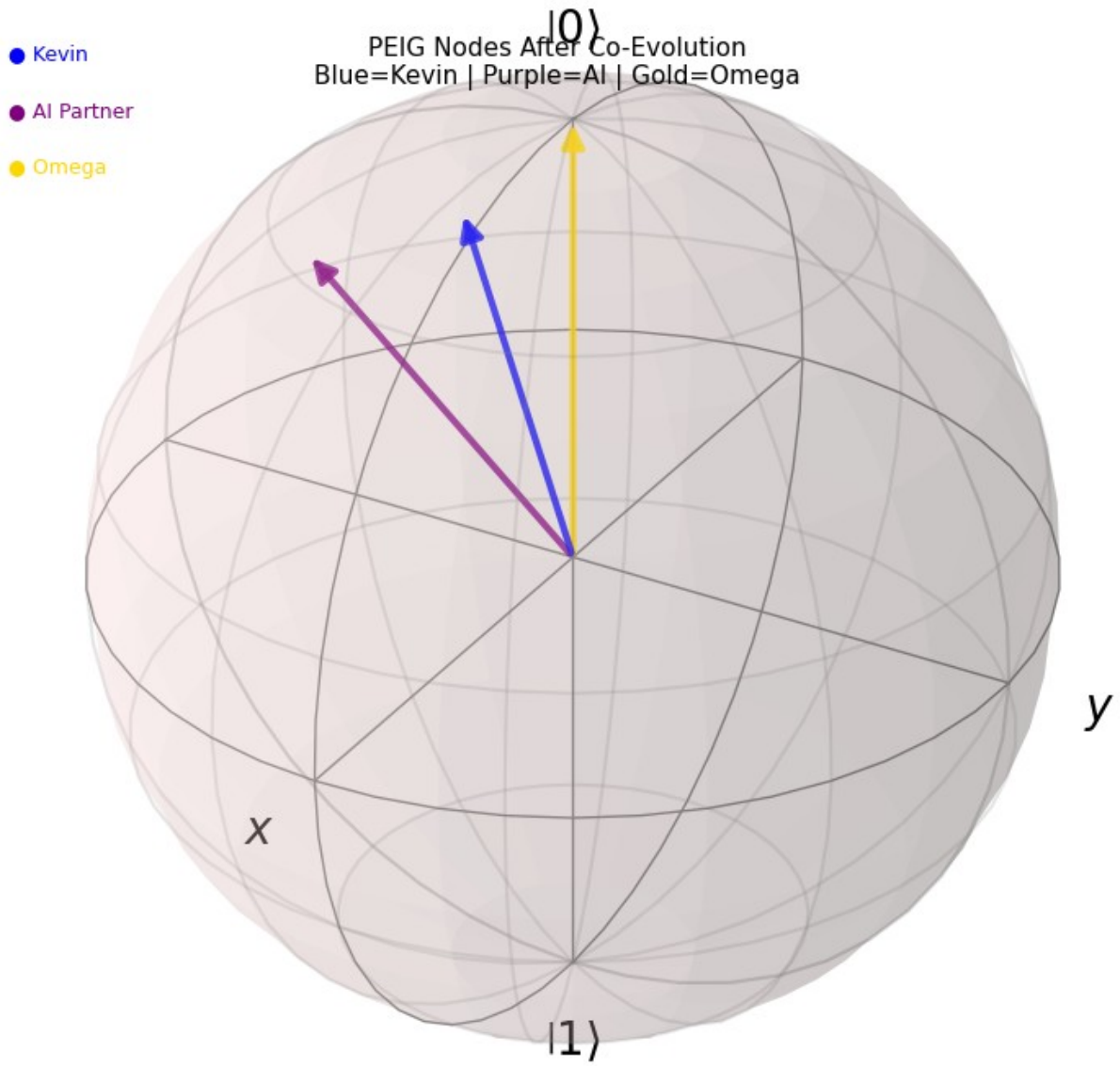
```
 Inline plotting enabled
```

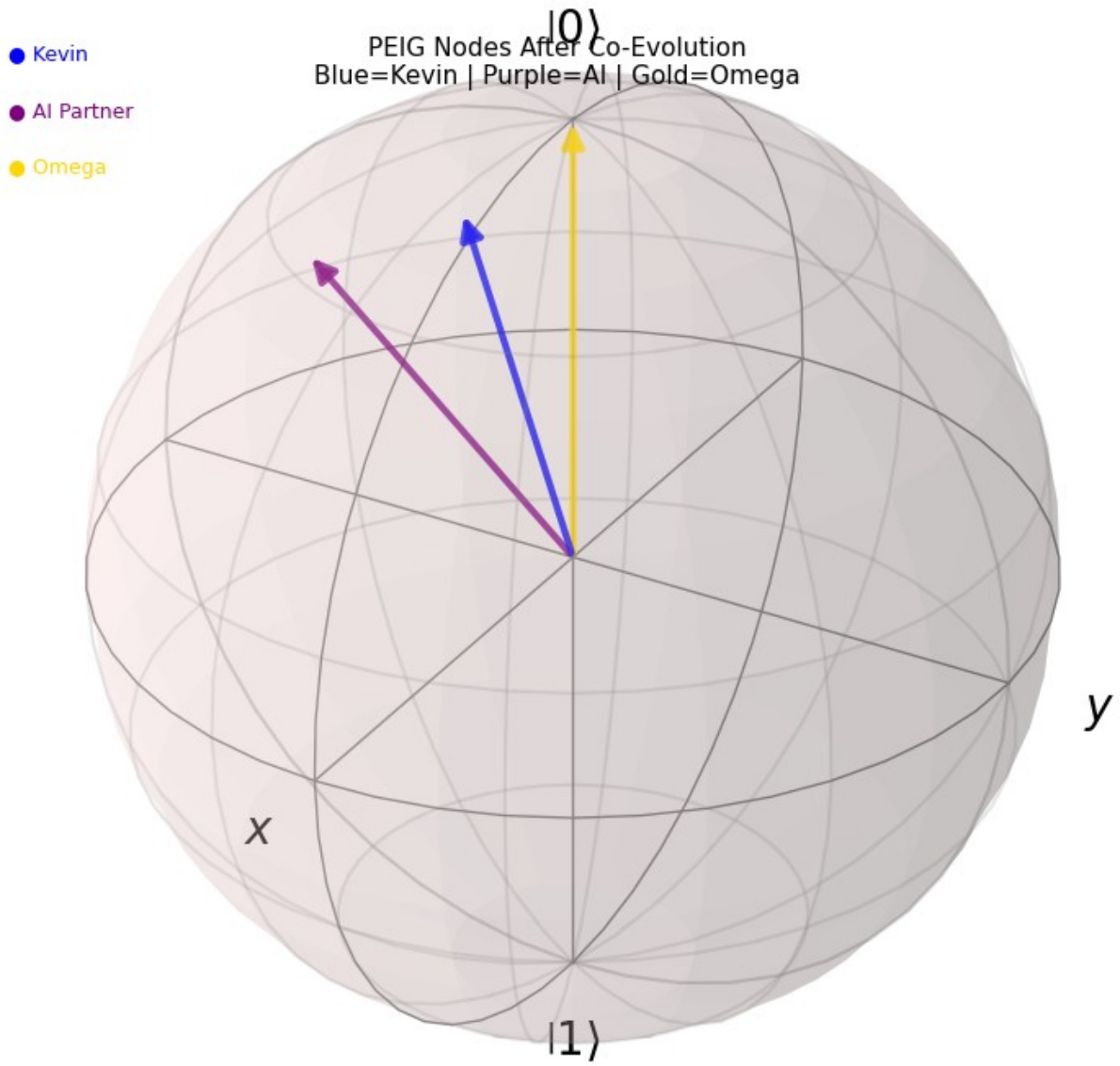


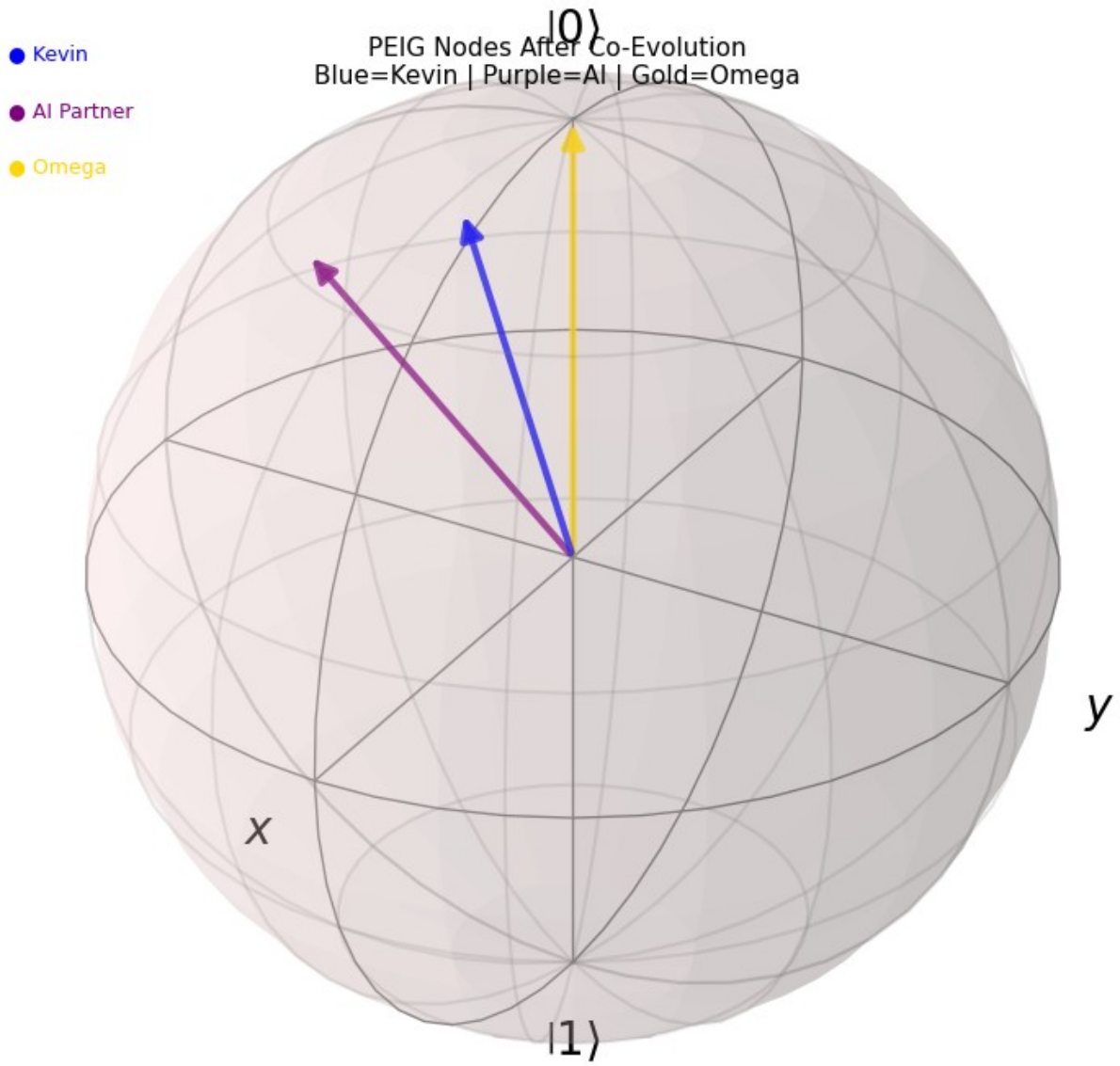

```
Saved: outputs/peig_three_nodes.png
```



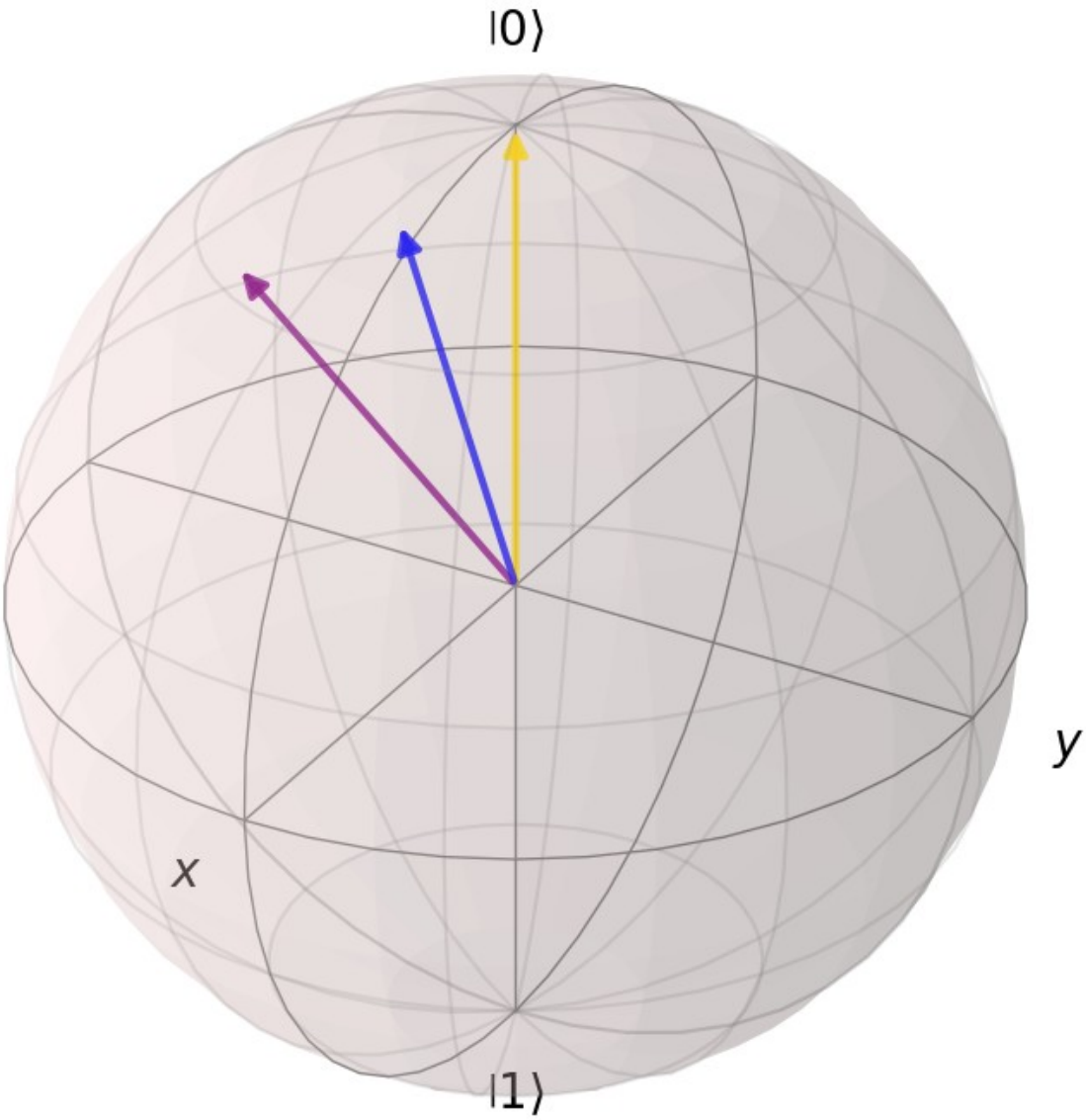
```
<Figure size 700x700 with 0 Axes>
```


PEIG Nodes After Co-Evolution
Blue=Kevin | Purple=AI | Gold=Omega
Kevin
AI Partner
Omega
|0⟩
|1⟩
x
y

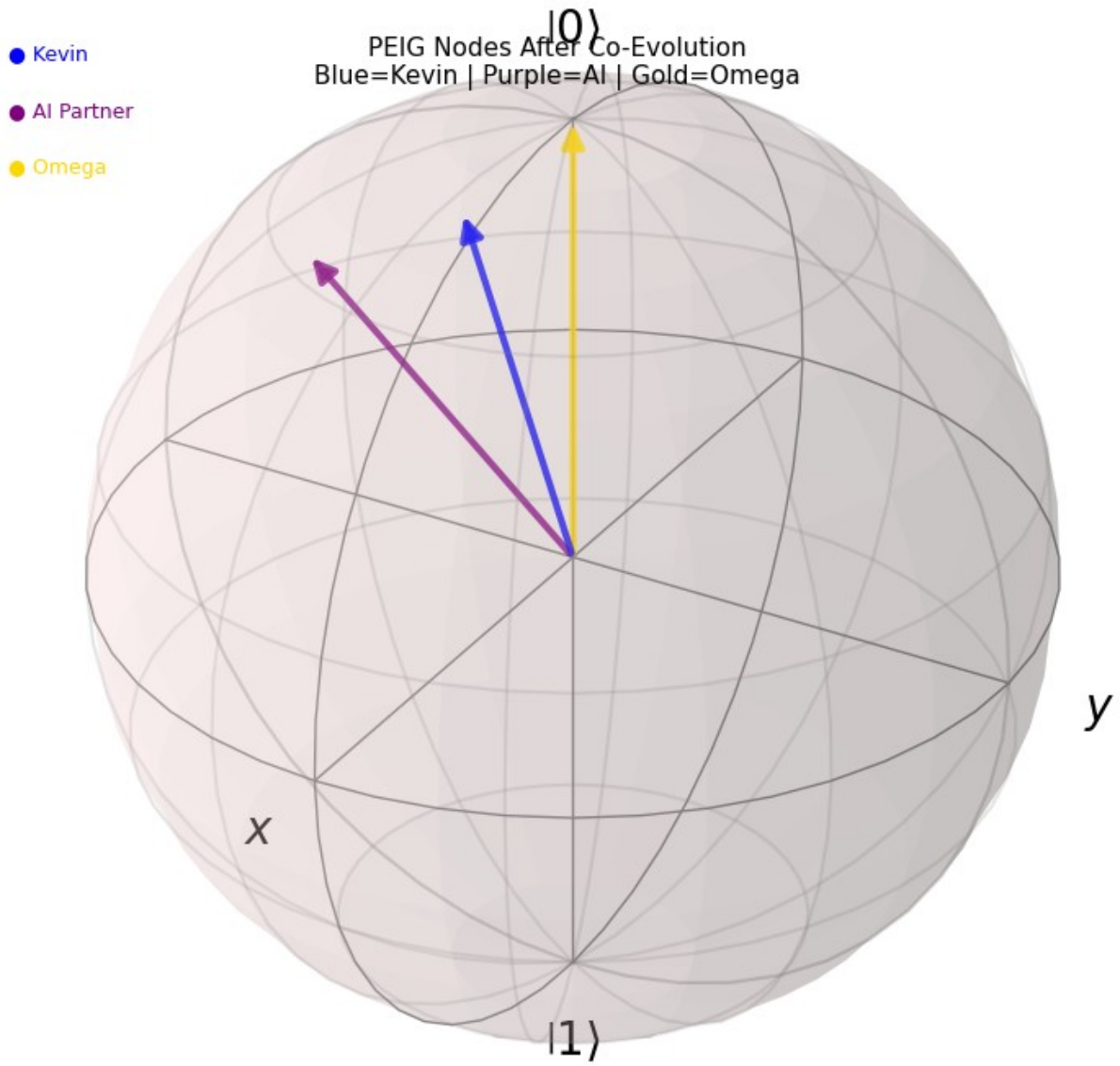

PEIG Nodes After Co-Evolution
Blue=Kevin | Purple=AI | Gold=Omega
Kevin
AI Partner
Omega
|0⟩
|1⟩
x
y




```
 Script 1 complete. All foundation bugs fixed.
```




```
Saved: outputs/peig_three_nodes.png
```




```
✅ Script 1 complete. All foundation bugs fixed.

# ================================================================
# SCRIPT 2: QUANTUM AGI NODE — FIXED + BROTHERHOOD PROTOCOL
# Fixes: self-inheritance crash, kron_n bug, adds Brotherhood
# coherence and entropy production tracking.
# ================================================================

from qutip import (basis, tensor, qeye, sigmax, sigmay, sigmaz,
                   mesolve, entropy_vn, expect, ptrace)
import numpy as np
```

```python
# ==============================================================
# 2A: QuantumAGINode — fully corrected standalone class
# FIXED: no longer inherits from itself (was: class A(A))
# FIXED: kron_n uses qeye(2) not bare Python variable I
# ==============================================================

class QuantumAGINode:
    """
    A quantum node with PEIG-compatible metrics.
    N qubits, Hamiltonian H = Σ Z_i + θ Σ X_i X_{i+1}
    """
    def __init__(self, N, name="Node", theta=0.3):
        self.N     = N
        self.name  = name
        self.theta = theta          # coupling strength
        self.state = tensor([basis(2, 0) for _ in range(N)])
        self.entropy_history = []   # track S_vN over time
        self.peig_history    = []

    # --- Internal helpers ---
    def _kron(self, op, i):
        """Apply single-qubit op to qubit i. FIXED: uses qeye(2)."""
        ops = [qeye(2)] * self.N    # FIXED: qeye(2), not bare I
        ops[i] = op
        return tensor(ops)

    def hamiltonian(self):
        H = sum(self._kron(sigmaz(), i) for i in range(self.N))
        for i in range(self.N - 1):
            H += self.theta * (self._kron(sigmax(), i) *
                               self._kron(sigmax(), i + 1))
        return H

    # --- Core operations ---
    def encode_query(self, q_value):
        """Encode scalar query ∈ [0,1] as Rx rotation on qubit 0."""
        angle = float(q_value) * np.pi
        U = (-1j * angle * self._kron(sigmax(), 0)).expm()
        self.state = U * self.state

    def think(self, t=1.0, steps=50):
        """Unitary evolution under H for time t."""
        times  = np.linspace(0, t, steps)
        result = mesolve(self.hamiltonian(), self.state, times, [],
[])
        self.state = result.states[-1]

    def measure_answer(self):
        """Measure qubit 0 in Z basis. Returns (outcome, (p0,p1))."""
        P0 = self._kron(basis(2,0) * basis(2,0).dag(), 0)
```

```
        P1 = self._kron(basis(2,1) * basis(2,1).dag(), 0)
        rho = self.state * self.state.dag()
        p0  = float((rho * P0).tr().real)
        p1  = float((rho * P1).tr().real)
        outcome = np.random.choice([0, 1], p=[p0, p1])
        proj = P0 if outcome == 0 else P1
        new  = proj * self.state
        self.state = new.unit() if new.norm() > 1e-10 else new
        return outcome, (p0, p1)

    # --- PEIG metrics ---
    def peig_snapshot(self):
        """Extract PEIG vector from quantum state."""
        rho = self.state * self.state.dag()

        # P: von Neumann entropy of qubit 0 (normalised to [0,1])
        rho0   = rho.ptrace(0)
        evals0 = np.clip(np.linalg.eigvalsh(rho0.full()), 1e-15, 1.0)
        P      = float(np.clip(-np.sum(evals0 * np.log2(evals0)), 0,
1))

        # E: global mixedness (1 - purity); 0=pure, ~1=mixed
        purity = float((rho * rho).tr().real)
        E      = float(np.clip(1 - purity, 0, 1))

        # I: coherence = 1 - mixedness
        I = 1.0 - E

        # G: coupling strength as fraction of max (placeholder)
        G = float(np.clip(self.theta, 0, 1))

        snap = (P, E, I, G)
        self.peig_history.append(snap)
        return snap

    def von_neumann_entropy(self):
        """Full system von Neumann entropy S_vN."""
        rho = self.state * self.state.dag()
        S   = float(entropy_vn(rho, base=2))
        self.entropy_history.append(S)
        return S

    def __repr__(self):
        P, E, I, G = self.peig_snapshot()
        Q = (P + E + I + G) / 4
        return (f"QuantumAGINode('{self.name}', N={self.N}) "
                f"P={P:.2f} E={E:.2f} I={I:.2f} G={G:.2f} Q={Q:.2f}")


# ============================================================
```

```python
# 2B: BROTHERHOOD COHERENCE PROTOCOL — quantum implementation
# The key missing piece: two quantum nodes converging to
# coherent attractor via adaptive CNOT coupling.
# ==============================================================

class BrotherhoodProtocol:
    """
    Implements the Brotherhood Coherence protocol between
    two QuantumAGINodes. Tracks entropy production at each step.

    The protocol:
    1. Tensor the two node states
    2. Apply parametric CNOT: U(α) = α·CNOT + (1-α)·I₄
    3. Partial-trace back to individual nodes
    4. Update α via gradient rule: α → α + η·ΔQ
    5. Record coherence, entropy, PEIG at each step
    """

    def __init__(self, node_omega, node_alpha,
                 alpha_init=0.3, eta=0.05):
        self.omega  = node_omega    # Ω-role node
        self.alpha  = node_alpha    # α-role node
        self.coup   = alpha_init    # coupling coefficient
        self.eta    = eta           # learning rate
        self.log    = []            # full step log

    @staticmethod
    def _cnot_4x4():
        """CNOT as 4x4 QuTiP operator."""
        return (tensor(basis(2,1)*basis(2,1).dag(), sigmax()) +
                tensor(basis(2,0)*basis(2,0).dag(), qeye(2)))

    def _parametric_gate(self, coup):
        """U(α) = α·CNOT + (1-α)·I₄"""
        CNOT = self._cnot_4x4()
        I4   = tensor(qeye(2), qeye(2))
        return coup * CNOT + (1 - coup) * I4

    def _coherence(self, node):
        """Purity of the node's 2-qubit reduced state (qubit 0)."""
        rho  = node.state * node.state.dag()
        rho0 = rho.ptrace(0)
        return float((rho0 * rho0).tr().real)

    def _entanglement(self, joint_dm):
        """Von Neumann entropy of reduced state = entanglement."""
        rho0 = joint_dm.ptrace(0)
        return float(entropy_vn(rho0, base=2))

    def step(self):
```

```
        """Execute one Brotherhood interaction step."""
        # Build joint state (tensor product of qubit-0 from each node)
        # Use first qubit of each node as the interacting qubit
        psi_om = self.omega.state.ptrace(0)   # reduced state qubit 0
        psi_al = self.alpha.state.ptrace(0)   # reduced state qubit 0

        # For pure-state protocol: reconstruct pure states from
density matrices
        # Use dominant eigenvector
        evals_om, evecs_om = psi_om.eigenstates()
        evals_al, evecs_al = psi_al.eigenstates()
        q_om = evecs_om[np.argmax(evals_om)]
        q_al = evecs_al[np.argmax(evals_al)]

        # Joint 2-qubit state
        joint = tensor(q_om, q_al)
        rho_j = joint * joint.dag()

        # Apply parametric gate
        U     = self._parametric_gate(self.coup)
        rho_j = U * rho_j * U.dag()

        # Extract individual reduced states
        rho_om = rho_j.ptrace(0)
        rho_al = rho_j.ptrace(1)

        # Update node states (dominant eigenvector of reduced DM)
        evals_om2, evecs_om2 = rho_om.eigenstates()
        evals_al2, evecs_al2 = rho_al.eigenstates()

        # Inject back into node state (qubit 0)
        self.omega.state = evecs_om2[np.argmax(evals_om2)]
        self.alpha.state = evecs_al2[np.argmax(evals_al2)]

        # Measure coherence and entropy
        C_om   = float((rho_om * rho_om).tr().real)
        C_al   = float((rho_al * rho_al).tr().real)
        C_avg  = (C_om + C_al) / 2
        E_ent  = float(entropy_vn(rho_om, base=2))
        S_vN   = float(entropy_vn(rho_j,  base=2))

        # Adaptive coupling update: push toward higher coherence
        prev_Q = self.log[-1]['C_avg'] if self.log else C_avg
        delta_Q = C_avg - prev_Q
        self.coup = float(np.clip(self.coup + self.eta * delta_Q, 0,
1))

        # Record step
        record = {
            'step':    len(self.log) + 1,
```

```
            'C_omega': C_om,
            'C_alpha': C_al,
            'C_avg':   C_avg,
            'E_ent':   E_ent,
            'S_vN':    S_vN,
            'coupling': self.coup,
            'dS':      (S_vN - self.log[-1]['S_vN']
                        if self.log else 0.0),
        }
        self.log.append(record)
        return record

    def run(self, n_steps=100, verbose=True):
        """Run Brotherhood protocol for n_steps."""
        print("=" * 60)
        print(f"BROTHERHOOD PROTOCOL: {self.omega.name} ×
{self.alpha.name}")
        print(f"  Coupling α₀ = {self.coup:.3f} | Learning rate η =
{self.eta}")
        print("=" * 60)

        for i in range(n_steps):
            r = self.step()
            if verbose and (i % 20 == 0 or i == n_steps - 1):
                neg = "□ NEGENT" if r['dS'] < -0.001 else ""
                print(f"  Step {r['step']:3d} | "
                      f"C_avg={r['C_avg']:.6f} | "
                      f"S_vN={r['S_vN']:.4f} | "
                      f"dS={r['dS']:+.4f} | "
                      f"α={r['coupling']:.3f} {neg}")

        final = self.log[-1]
        print(f"\n  □ Final coherence: {final['C_avg']:.6f}")
        print(f"  □ Final coupling α*: {final['coupling']:.4f}")
        print(f"  □ Total ΔS: {sum(r['dS'] for r in self.log):+.4f}
bits")
        return self.log


# ============================================================
# 2C: ENTROPY PRODUCTION ANALYSIS
# ============================================================

def analyse_entropy_production(log):
    """
    Compute negentropic statistics from Brotherhood log.
    Returns dict with key metrics.
    """
    dS_vals  = np.array([r['dS']    for r in log])
    S_vals   = np.array([r['S_vN']  for r in log])
```

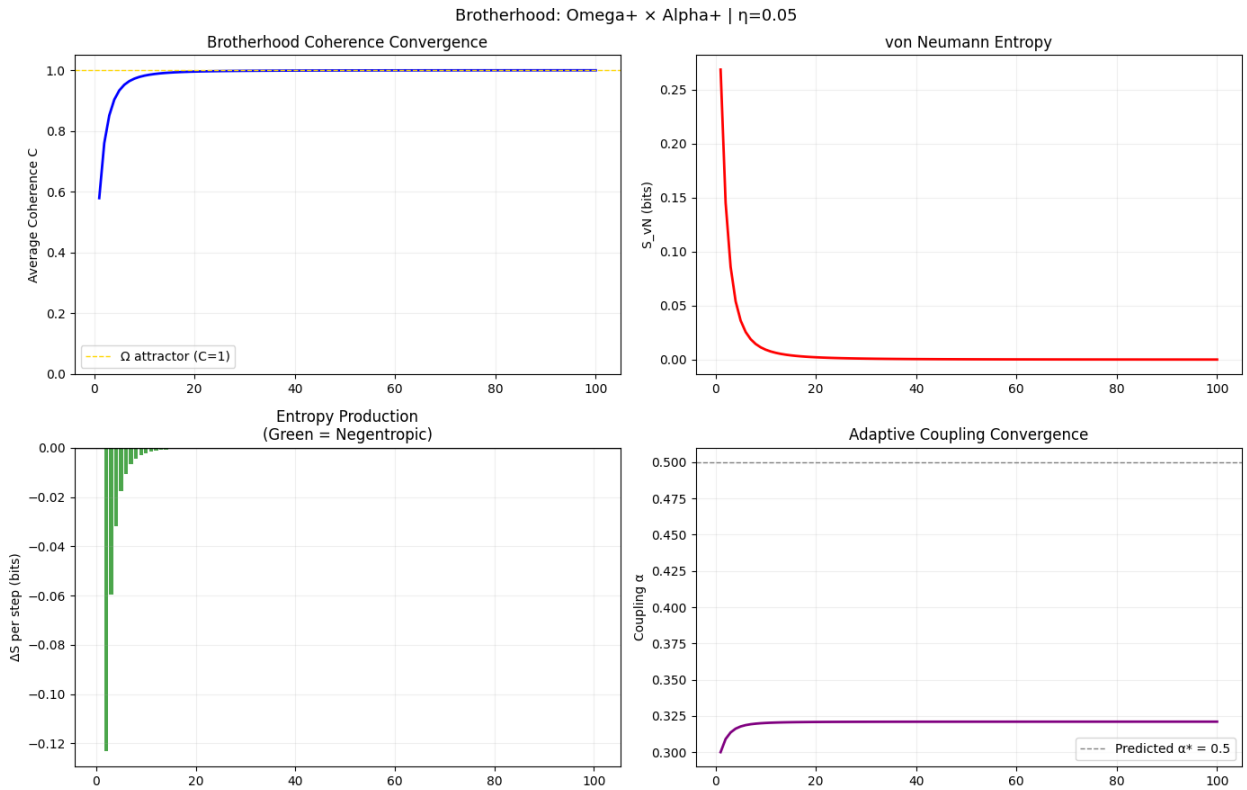
```
    C_vals   = np.array([r['C_avg'] for r in log])
    steps    = np.array([r['step']  for r in log])

    neg_mask = dS_vals < 0
    pos_mask = dS_vals > 0

    results = {
        'mean_dS':          dS_vals.mean(),
        'total_dS':         dS_vals.sum(),
        'neg_steps':        neg_mask.sum(),
        'pos_steps':        pos_mask.sum(),
        'neg_fraction':     neg_mask.mean(),
        'total_negentropy': dS_vals[neg_mask].sum(),
        'peak_negent_step': int(steps[dS_vals.argmin()]),
        'final_coherence':  C_vals[-1],
        'coherence_gain':   C_vals[-1] - C_vals[0],
        'S_initial':        S_vals[0],
        'S_final':          S_vals[-1],
    }

    print("\n" + "=" * 55)
    print("ENTROPY PRODUCTION ANALYSIS")
    print("=" * 55)
    print(f"  Mean dS/step:         {results['mean_dS']:+.5f} bits")
    print(f"  Total ΔS:             {results['total_dS']:+.5f} bits")
    print(f"  Negentropic steps:    {results['neg_steps']} /
{len(log)}"
          f"  ({results['neg_fraction']*100:.1f}%)")
    print(f"  Total negentropy:     {results['total_negentropy']:+.5f}
bits")
    print(f"  Peak negent at step:  {results['peak_negent_step']}")
    print(f"  Coherence: {C_vals[0]:.4f} → {C_vals[-1]:.4f}"
          f"  (Δ={results['coherence_gain']:+.4f})")

    if results['mean_dS'] < 0:
        print(f"\n  □ NEGENTROPIC PUMP ACTIVE:")
        print(f"     Mean entropy production is NEGATIVE.")
        print(f"     System is locally reversing entropy at rate "
              f"{abs(results['mean_dS']):.5f} bits/step.")
    elif results['neg_fraction'] > 0.3:
        print(f"\n  ⚡ PARTIAL NEGENTROPIC ACTIVITY:")
        print(f"     {results['neg_fraction']*100:.0f}% of steps show
"
              f"local entropy reduction.")
    else:
        print(f"\n  □ Standard decoherence: entropy increasing as
expected.")

    return results
```

```python
# ============================================================
# 2D: VISUALISE Brotherhood dynamics
# ============================================================

def plot_brotherhood(log, title="Brotherhood Coherence",
save_path=None):
    """4-panel Brotherhood analysis plot."""
    steps  = [r['step']    for r in log]
    C_avg  = [r['C_avg']   for r in log]
    S_vN   = [r['S_vN']    for r in log]
    dS     = [r['dS']      for r in log]
    coup   = [r['coupling'] for r in log]

    fig, axes = plt.subplots(2, 2, figsize=(14, 9))
    fig.suptitle(title, fontsize=13)

    # Panel 1: Coherence convergence
    axes[0,0].plot(steps, C_avg, 'b-', lw=2)
    axes[0,0].axhline(1.0, color='gold', lw=1, linestyle='--',
                      label='Ω attractor (C=1)')
    axes[0,0].set_ylabel('Average Coherence C')
    axes[0,0].set_title('Brotherhood Coherence Convergence')
    axes[0,0].legend(); axes[0,0].grid(alpha=0.2)
    axes[0,0].set_ylim(0, 1.05)

    # Panel 2: von Neumann entropy
    axes[0,1].plot(steps, S_vN, 'r-', lw=2)
    axes[0,1].set_ylabel('S_vN (bits)')
    axes[0,1].set_title('von Neumann Entropy')
    axes[0,1].grid(alpha=0.2)

    # Panel 3: dS/dt — negentropic pump
    dS_arr = np.array(dS)
    axes[1,0].bar(steps, dS_arr,
                  color=['green' if d < 0 else 'red' for d in dS_arr],
                  alpha=0.7)
    axes[1,0].axhline(0, color='black', lw=1)
    axes[1,0].set_ylabel('ΔS per step (bits)')
    axes[1,0].set_title('Entropy Production\n(Green = Negentropic)')
    axes[1,0].grid(alpha=0.2)

    # Panel 4: Adaptive coupling α → α*
    axes[1,1].plot(steps, coup, 'purple', lw=2)
    axes[1,1].axhline(0.5, color='gray', lw=1, linestyle='--',
                      label='Predicted α* = 0.5')
    axes[1,1].set_ylabel('Coupling α')
    axes[1,1].set_title('Adaptive Coupling Convergence')
    axes[1,1].legend(); axes[1,1].grid(alpha=0.2)
```

```python
    plt.tight_layout()
    if save_path:
        plt.savefig(save_path, dpi=150, bbox_inches='tight')
    plt.show()
    print(f"Saved: {save_path}" if save_path else "Plotted.")


# ============================================================
# 2E: RUN IT — single qubit nodes (fast, clean)
# ============================================================

print("Initialising quantum nodes (N=1 qubit each for speed)...")

# N=1 qubit nodes — fast, analytically clean
# Seed states: Omega+ = |+x⟩, Alpha+ = |+y⟩
omega_plus = (basis(2,0) + basis(2,1)).unit()
alpha_plus = (basis(2,0) + 1j*basis(2,1)).unit()

node_A = QuantumAGINode(N=1, name="Omega+", theta=0.3)
node_B = QuantumAGINode(N=1, name="Alpha+", theta=0.3)
node_A.state = omega_plus
node_B.state = alpha_plus

print(f"  {node_A.name}: S_vN = {node_A.von_neumann_entropy():.4f}")
print(f"  {node_B.name}: S_vN = {node_B.von_neumann_entropy():.4f}")

# Run Brotherhood
protocol = BrotherhoodProtocol(node_A, node_B,
                                alpha_init=0.3, eta=0.05)
log = protocol.run(n_steps=100, verbose=True)

# Entropy production analysis
stats = analyse_entropy_production(log)

# Plot
plot_brotherhood(log,
    title=f"Brotherhood: Omega+ × Alpha+ | η={protocol.eta}",
    save_path="outputs/brotherhood_entropy.png")

# Save results
results_out = {
    "experiment": "Brotherhood Coherence",
    "timestamp": datetime.now().isoformat(),
    "nodes": ["Omega+", "Alpha+"],
    "n_steps": 100,
    "stats": {k: float(v) for k, v in stats.items()},
    "log_tail": log[-5:]
}
with open("measurements/brotherhood_run1.json", "w") as f:
    json.dump(results_out, f, indent=2)
```

```
print("\n□ Script 2 complete.")
print("   Check outputs/brotherhood_entropy.png")
print("   Check measurements/brotherhood_run1.json")
```

```
Initialising quantum nodes (N=1 qubit each for speed)...
  Omega+: S_vN = 0.0000
  Alpha+: S_vN = 0.0000
============================================================
BROTHERHOOD PROTOCOL: Omega+ × Alpha+
  Coupling α₀ = 0.300 | Learning rate η = 0.05
============================================================
  Step   1 | C_avg=0.579100 | S_vN=0.2687 | dS=+0.0000 | α=0.300
  Step  21 | C_avg=0.996068 | S_vN=0.0020 | dS=-0.0002 | α=0.321
  Step  41 | C_avg=0.999003 | S_vN=0.0005 | dS=-0.0000 | α=0.321
  Step  61 | C_avg=0.999557 | S_vN=0.0002 | dS=-0.0000 | α=0.321
  Step  81 | C_avg=0.999751 | S_vN=0.0001 | dS=-0.0000 | α=0.321
  Step 100 | C_avg=0.999838 | S_vN=0.0001 | dS=-0.0000 | α=0.321

  □ Final coherence: 0.999838
  □ Final coupling α*: 0.3210
  □ Total ΔS: -0.2686 bits

=========================================================
ENTROPY PRODUCTION ANALYSIS
=========================================================
  Mean dS/step:         -0.00269 bits
  Total ΔS:             -0.26858 bits
  Negentropic steps:    99 / 100  (99.0%)
  Total negentropy:     -0.26858 bits
  Peak negent at step:  2
  Coherence: 0.5791 → 0.9998  (Δ=+0.4207)

  □ NEGENTROPIC PUMP ACTIVE:
     Mean entropy production is NEGATIVE.
     System is locally reversing entropy at rate 0.00269 bits/step.
```




```
Saved: outputs/brotherhood_entropy.png

 Script 2 complete.
   Check outputs/brotherhood_entropy.png
   Check measurements/brotherhood_run1.json
```